VERIQO Sisa Backlogs — Closure Report

Round: System Integration Proof **Branch:** `claude/l99-gap-coverage-nv70zy` **Date:** 2026-09-03 **Source:** *Sisa Backlogs*

---

0. The headline

The Sisa Backlogs ended by naming the milestone:

> The five VERIQO fabrics are not merely co-existing components; they constitute one executable, governed evidence-to-proof operating system.

That claim is now executable, and it passes — for all six domains. `TestSystemIntegrationProofForEveryDomain` runs the whole chain (CASE → FORWARD → EVIDENCE → INTELLIGENCE → TRUST → EQF → PROOF → REVERSE → FINDING → AUTHORIZED DECISION → LEDGER → REPLAY) through Maritime, Commodity, Supply Chain, Insurance, Trade Finance and Dispute Evidence Support, holding four properties throughout: no bypass, no duplicate engine, no synthetic promotion, fail-closed.

And the evidence in every one of those runs is a fixture. The chain executes and records; it has not been run on real rights-aware commercial data. That is `LIVE_DATA`, it stays `BLOCKED_EXTERNAL`, and this round did not move it.

Both sentences are true, and reporting either without the other would be the overstatement this codebase is arranged to prevent.

---

1. Item-by-item closure

#	Sisa Backlogs item	Verdict	Where
1	Three-level proof vocabulary: `PROOF_OBJECT_CREATED` ≠ `PROOF_QUALIFIED` ≠ `PROOF_EXTERNALLY_ATTESTED`	**CLOSED**	`pkg/proof/attestation.go`
2	Case Fabric: nine attributes per phase; semantic spine, not a workflow engine	**CLOSED**	`pkg/casefabric/phasecontract.go`
3	`RUNTIME_EVIDENCE_REF` — article → code → test → *actual runtime evidence*	**CLOSED**	`pkg/assurance`, `cmd/veriqo-runtime-evidence`, `evidence/RUNTIME_EVIDENCE.json`
4	40 canonical objects: nine properties each, or "40" is just a number	**CLOSED**	`pkg/ontology/contract.go`
5A	External proof — the missing external dependencies	**CLOSED as blockers** (8 → 10, none upgraded)	`BLOCKER_REGISTER.json`
5B	Real-world evidence — AIS, SAR, EO, BoL, Customs, Insurance, P&I, Port, Weather, Sanctions	**DOCUMENTED, NOT CLOSED**	`docs/architecture/REAL_WORLD_EVIDENCE_GAP.md`
5C	Domain semantics: ontology, evidence, obligations, rules, outcome — no duplicate engines	**CLOSED**	`pkg/casefabric/semantics.go`
6	VERIQO Case Proof Graph	**CLOSED**	`pkg/caseproofgraph`
7	**VERIQO System Integration Proof**	**CLOSED**	`test/integration/system_integration_proof_test.go`

---

2. Item 1 — "Proof Object" is not "proof"

The most available way for this system to overstate itself was to let two different words be one. A Proof Object is a *container*: it exists the moment somebody assembles one, and assembling one says nothing about the contents.

Three levels, with the third unreachable from inside:

Level	Means	Reachable by VERIQO alone?
`PROOF_OBJECT_CREATED`	components assembled, object sealed	yes — it is the zero value
`PROOF_QUALIFIED`	the evidence satisfies VERIQO's own qualification rules	yes — **this is the ceiling**
`PROOF_EXTERNALLY_ATTESTED`	a named outside party examined it and agreed the procedure was correctly applied	**never**

Four properties make the distinction hold rather than merely exist:

- **`AttestationLevel` is derived, never stored.** `LevelOf` computes it. The field that decides whether the word "proof" may be used is not one anybody can write.
- **`IsProof()` returns true only at level three.** A report saying "VERIQO has proved" is checkable against the type.
- **A self-run attestor is refused.** `TestVeriqoCannotAttestToItself`. So is a party to the matter.
- **No VERIQO code path calls `RaiseToExternallyAttested`.** `TestNoVeriqoCodePathReachesLevelThree`. It exists so that the day a real assessor produces a real statement there is a typed place to put it — and so that until that day, the level cannot be reached by accident.

The gap this creates is now a named blocker: `INDEPENDENT_ASSURANCE`.

---

3. Item 2 — the phase contract

Nine phases were a list. A list is what a workflow engine has, and the risk was the Case Fabric becoming one under a new name, sitting beside `pkg/workflow` doing the same job twice.

Every phase now declares nine things: **state, entry condition, exit condition, required evidence, blocking evidence, authority, owner, failure state, replay reference.** A blank in any of them fails the build.

Two of the nine do the real work:

- **Blocking evidence** is the attribute a workflow engine does not have. It is *knowledge that stops work* — an unresolved material contradiction, an unassessed source set, a material AI contribution nobody reviewed — not a gate somebody failed to open.
- **Failure state** is never empty. A phase with nowhere to fail to fails open, which is the one thing the fabric may not do.

The line between spine and engine is held by four tests, not by a paragraph:

Test	Holds
`TestPhaseContractIsSemanticNotProcedural`	no entry/exit condition may describe a procedural act — clicking, approving, a ticket moving, a queue draining
`TestCaseFabricDoesNotImportTheWorkflowEngine`	no import of `pkg/workflow`, `pkg/execution` or `pkg/kernel/execgraph`
`TestTheFabricHasNoExecutionVocabulary`	no `Run`, `Execute`, `Schedule`, `Dispatch`, `Enqueue` or `Retry` on `Case`
`TestContractExitConditionsMatchTheCode`	the stated exit conditions are the ones the engine actually enforces

> **Case Fabric** — the semantic spine: what is known, and what that means. **Workflow** — the execution mechanism: who does what, when, in what order.

---

4. Item 3 — runtime evidence

The backlog's point was exact: *article → code → test* is not enterprise assurance. A test proves a control behaves correctly when exercised deliberately. It says nothing about whether the control ran in the system as assembled, or left anything behind when it did.

So `cmd/veriqo-runtime-evidence` **executes the canonical chain** and records what it emitted. Ten real audit events:

```
AUDIT-001-case.opened            AUDIT-006-case.hypothesis_tested
AUDIT-002-case.scoped            AUDIT-007-case.qualification_begun
AUDIT-003-case.evidence_pinned   AUDIT-008-case.proof_attached
AUDIT-004-case.hypothesis_recorded  AUDIT-009-case.resolved
AUDIT-005-case.claim_registered  AUDIT-010-proof.sealed
```

The matrix cites those ids, and `TestEveryRuntimeEvidenceRefResolves` **fails if a row cites a record the run did not emit**. Without that resolution check, `RUNTIME_EVIDENCE_REF` would be a free-text field — the "spreadsheet of good intentions" the rest of the matrix exists to avoid.

The new link made the audit stricter, and the numbers went down. Three articles that previously read `EXTERNAL_QUALIFICATION` now read `ASSURANCE_GAP`, because they are tested and designed to leave a record but no executed run has produced one:

	Before	After
`OPEN`	3	3
`INTEGRATION_GAP`	0	0
`ASSURANCE_GAP`	18	**21**
`EXTERNAL_QUALIFICATION`	9	**6**
`QUALIFIED`	0	0

That is the column working. A traceability chain that got *easier* to satisfy after adding a requirement would not have been measuring anything.

The run is deterministic — logical ticks, fixed identifiers, no wall-clock time, no randomness — so the citations are stable. The artefact states its own boundary: the evidence is a fixture, and `TestTheArtefactStatesItsOwnBoundary` fails if that disclosure is removed.

---

5. Item 4 — forty objects, or forty names

Agreed, and acted on: the count was never the achievement. Every one of the 40 registered object types now declares nine things — **object id, schema version, canonicalization, content hash, provenance, authority, lifecycle, mutation rule, replay behaviour** — and `ValidateObjectContracts` refuses both directions of drift: a type with no contract (a name), and a contract for a type nobody registered (documentation of something that does not exist).

Five properties are asserted across all 40:

- **Hash semantics are consistent** — every object canonicalizes as RFC 8785 JCS with the hash field excluded from its own computation. Forty different canonicalizations would mean forty different hashes for the same bytes.
- **Every mutation rule constrains something.** A rule that permits anything is not a rule, and `TestEveryObjectDeclaresAMutationRule` rejects one.
- **Every object is reproducible** from the record by a party that does not trust the runtime.
- **Provenance and authority are separate answers.** Where an instance came from is not who was entitled to put it there, and in a dispute the second question is usually the one that matters.
- **Every object names an owning package.**

Writing this contract found nine genuinely thin declarations in my own first draft — `"acquisition record"`, `"onboarding record"`, `"the parties"`, `"status transitions are events"` — each of which read as a specification and specified nothing. All nine were rewritten.

---

6. Item 5 — the three big remaining works

5A · External proof — the register was understating the dependency

The backlog named seven external items. Five were already in `BLOCKER_REGISTER.json`. **Two were not**, which meant the register understated what VERIQO depends on outsiders for:

Added	Why it is genuinely external
`EXTERNAL_TSA`	VERIQO holds no TSA relationship. Its self-hosted facility is a tamper-evident chain and proves nothing about wall-clock time. The typed landing place exists; **no code change is needed to accept a real token**
`INDEPENDENT_ASSURANCE`	No assessor has examined any control. Until one has, no article can reach `QUALIFIED` and no proof object can be described as proved

The register grew from 8 to 10. Not one of the original eight changed status. Three remain `BLOCKED_EXTERNAL`, five remain `READY_FOR_EXTERNAL_QUALIFICATION`, exactly as they stood.

5B · Real-world evidence — documented, not closed

Six domains now declare **35 evidence classes**, each naming a real source. Every one is a fixture.

14 of the 35 are party-mediated — acquired through a party to the matter rather than independently. That is not a defect (a bill of lading is the carrier's document; there is no independent copy) but it is a fact about what those classes can establish, and `TestEveryDomainHasPartyMediatedEvidence` fails for any domain claiming otherwise, because for a real matter that claim is not credible.

`docs/architecture/REAL_WORLD_EVIDENCE_GAP.md` sets out what acquisition actually requires per source family, and makes one point worth repeating here: **a real connector is not an HTTP client.** It is a licence encoded as rights, an authority check that runs before contact, a provenance record, an independence assessment, and a retention position. All five mechanisms exist. What is missing is a licence to run them against.

5C · Domain semantics — closed, as data

Each of the six domains declares its ontology, evidence classes, proof obligations, rules and outcome vocabulary. **All five as data.**

That is the load-bearing constraint, not a stylistic one: a domain that needed its own engine to express its semantics would have forked the fabric. `TestDomainSemanticsAreDataNotEngines` fails on any import of a domain package, and on any comparison or switch against a specific domain constant — a lookup by parameter is fine, branching on `DomainMaritime` is an engine growing inside a data table.

Every obligation states what would **falsify** it. An obligation with no falsifier is not a test, and a domain built on untestable obligations proves nothing.

---

7. Item 6 — the Case Proof Graph

One canonical object holding everything behind a case: CASE → CLAIM → PROOF OBJECT (with its nine sub-nodes) plus TIMELINE, ACTORS, ENTITIES, EVENTS, DECISIONS and OUTCOME.

The five adjectives the backlog asked for are each an enforced property:

Property	How
canonical	closed node and edge vocabularies; every node kind resolves to a registered ontology type that has an object contract
hashed	each node hashes its content, each edge its endpoints, the root hashes both sets — any change anywhere changes the root
auditable	the root is emitted to the one audit ledger
replayable	`VerifyGraph` recomputes every hash from the content
rights-aware	`Project` returns a real subgraph, sealed and verifiable on its own

Projection is not redaction, and the difference is the point. A redacted view still *contains* what it hides, and anything that contains it can leak it. A projection never held the excluded nodes at all — and edges to withheld nodes are dropped, because an edge to a node you cannot see tells you that node exists and what relation it bears, which is often the whole disclosure.

Building this surfaced a real design error in my own first draft: `Project` passed a content hash where `access.Evaluate` expects an evidence version id, so every projection silently withheld everything. A disclosure grant names a *version*, not a hash of bytes. Fixed by specializing the grant per node, and by separating the two questions honestly — evidence nodes go to `pkg/disclosure/access` as the sole authority; structural nodes are governed by the two levels and the right, because inventing a fake evidence version id to route them through would be worse than no decision.

---

8. Item 7 — the System Integration Proof

The centrepiece. One test, six subtests, the whole chain per domain.

What each run proves:

Property	Enforced by
no bypass	a finding is attempted before `TRUST` and refused; self-authorization is attempted and refused
no duplicate engine	`VerifyAgainstContract` — every stage ran in the package the contract binds it to
no synthetic promotion	the sealed object reaches `PROOF_QUALIFIED` and `IsProof()` is asserted false
fail-closed	an adjudicatory decision is attempted and refused, in every domain; an invented domain state is attempted and refused

Plus, per domain: the domain's declared semantics validate; its own state projects onto the canonical phase; the two FREF directions **close** over the same evidence; the Case Proof Graph builds, verifies and projects; everything lands in the one ledger; and **five independent re-verifications** pass — audit chain, canonical event chain, case timeline, proof object, temporal chain.

Every run ends by asserting the honest limits survive: the fixture limitation is carried into the case outcome, and the proof object still reports no external qualification.

`TestNoDomainHasItsOwnChain` makes the anti-duplication claim explicit rather than leaving it implied by six passing subtests, and fails if the proof stops covering every registered domain.

---

9. Verification

`scripts/verify.sh`, all six stages: build, vet, gofmt, the zero-external-dependency invariant, the full suite under `-race`, and race-repeat 5× on consensus-critical packages.

Package	Tests	Coverage
`pkg/proof`	66	81.9%
`pkg/casefabric`	57	79.8%
`pkg/assurance`	54	87.4%
`pkg/ontology`	32	91.5%
`pkg/caseproofgraph`	23	82.9%
`test/integration`	73	—

New this round: `pkg/caseproofgraph`, `cmd/veriqo-runtime-evidence`, `pkg/proof/attestation.go`, `pkg/casefabric/phasecontract.go`, `pkg/casefabric/semantics.go`, `pkg/ontology/contract.go`, the runtime-evidence column, and the system integration proof.

Structural counts, each asserted by test: 40 object types / 40 contracts · 9 phases / 9 phase contracts · 6 domains / 6 semantics declarations · 35 evidence classes (14 party-mediated) · 10 external blockers.

---

10. Claims this round does not support

1. **That any of this works on real data.** Every run is a fixture. `LIVE_DATA` remains `BLOCKED_EXTERNAL`.
2. **That anything is proved.** The ceiling reached is `PROOF_QUALIFIED`. `INDEPENDENT_ASSURANCE` is now a named blocker precisely because nothing crosses to `PROOF_EXTERNALLY_ATTESTED`.
3. **That any article is `QUALIFIED`.** 0 of 30, and the count went the honest direction this round rather than the flattering one.
4. **That any evidence carries an independent temporal attestation.** `EXTERNAL_TSA` is now a named blocker.
5. **That the eight pre-existing external gates moved.** They did not. Two were added; none was upgraded.
6. **That the domain semantics are correct.** They are declared and internally consistent. No practitioner in any of the six domains has reviewed them.
7. **That the Case Proof Graph is admissible, useful, or wanted** by any tribunal. None has been offered one.

---

11. Status per MIP §34

```
Three-level proof vocabulary ...... IMPLEMENTED · UNIT_TESTED · ADVERSARIAL_TESTED
Phase contract (9 x 9) ............ IMPLEMENTED · UNIT_TESTED
Runtime evidence column ........... IMPLEMENTED · UNIT_TESTED · INTEGRATION_TESTED
Canonical object contract (40 x 9)  IMPLEMENTED · UNIT_TESTED
Case Proof Graph .................. IMPLEMENTED · UNIT_TESTED · ADVERSARIAL_TESTED
Domain semantics (6 domains) ...... IMPLEMENTED · UNIT_TESTED
System Integration Proof .......... IMPLEMENTED · INTEGRATION_TESTED (fixtures)
External blocker register ......... IMPLEMENTED (10 entries, 0 upgraded)
Real-world evidence ............... DESIGNED · BLOCKED_EXTERNAL (LIVE_DATA)
External attestation .............. DESIGNED · BLOCKED_EXTERNAL (EXTERNAL_TSA, INDEPENDENT_ASSURANCE)
```

No line reads DONE.

---

12. What is left

Not engineering. The Sisa Backlogs' own closing assessment was right: what remains is not building VERIQO but taking what is built **outside the repository** to obtain external qualification and real-world evidence.

Three engagements, in the order that unblocks the most:

1. **An independent assessor** → unblocks `INDEPENDENT_ASSURANCE`, and with it the first `QUALIFIED` article in the constitutional proof audit.
2. **A commercial data agreement** → unblocks `LIVE_DATA`, and turns 35 declared evidence classes into 35 real ones.
3. **A TSA relationship** → unblocks `EXTERNAL_TSA`. The typed landing place already exists; no code change is required to accept a real token.

None of the three is a development task.

Appendix — the audit, the phase contracts and the object contracts, as generated

```
ART  VERDICT                CONTROL
-------------------------------------------------------------------------------------------------
1    EXTERNAL_QUALIFICATION  Finding requires a sealed proof object with a pinned evidence set
2    ASSURANCE_GAP           Acquisition records provenance without conferring qualification
3    ASSURANCE_GAP           Transitive source clustering: same-root data counts once
4    ASSURANCE_GAP           Rights are evaluated before contact, not after
5    EXTERNAL_QUALIFICATION  Raw bytes are preserved before any transformation
6    EXTERNAL_QUALIFICATION  A finalized version is structurally unupdatable
7    ASSURANCE_GAP           Historical cases resolve against their historical policy version
8    ASSURANCE_GAP           AI cannot create, alter, qualify or sign evidence
9    OPEN                    A zero-knowledge proof establishes only its stated predicate
10   ASSURANCE_GAP           Replay is verifiable without trusting the runtime
11   ASSURANCE_GAP           Dissent is carried through qualification, never deleted
12   ASSURANCE_GAP           The same policy applies to every party absent an authorized exc...
13   ASSURANCE_GAP           Party influence on acquisition is recorded
14   ASSURANCE_GAP           Conflicts are declared rather than concealed
15   OPEN                    No differential benefit from a dispute outcome
```

```
16   EXTERNAL_QUALIFICATION  The platform does not determine legal liability
17   ASSURANCE_GAP           Redaction never modifies the original
18   OPEN                    Redacted content is absent from the derivative's bytes
19   ASSURANCE_GAP           VERIQO enforces privilege; it does not determine it
20   ASSURANCE_GAP           View, export, AI processing and training are separate grants
21   ASSURANCE_GAP           A redacted derivative must still pass AI policy
22   EXTERNAL_QUALIFICATION  Every derivative is a new immutable version
23   EXTERNAL_QUALIFICATION  Process evidence is itself evidence
24   ASSURANCE_GAP           Every disclosure emits a ledger event
25   ASSURANCE_GAP           Privilege transitions are immutable events
26   ASSURANCE_GAP           Policy change is never quietly applied to history
27   ASSURANCE_GAP           Material AI contribution is recorded and human-reviewed
28   ASSURANCE_GAP           UNKNOWN independence is never treated as INDEPENDENT
29   ASSURANCE_GAP           OBSERVED_ABSENT only after the observability gate
30   ASSURANCE_GAP           Integrity, provenance, qualification, neutrality and legal dete...
-----------------------------------------------------------------------------------------------
30 articles: 3 OPEN, 0 INTEGRATION_GAP, 21 ASSURANCE_GAP, 6 EXTERNAL_QUALIFICATION, 0 QUALIFIED. QUALIFIED requires an outside party to ha

=== OPENED ===
  ENTRY              a case identity exists: case id, tenant and a registered domain
  EXIT               the matter, jurisdiction and time window are fixed and a mission is stated
  REQUIRED EVIDENCE  a registered domain projection for the opening domain
  BLOCKING EVIDENCE  an existing open case over the same matter and tenant, which would split one dispute into two records
  AUTHORITY          any actor with case-open authority in the opening domain
  OWNER              the opening domain
  FAILURE STATE      CLOSED
  REPLAY             casefabric timeline entry kind=case_opened; VerifyTimeline re-derives the chain

=== SCOPED ===
  ENTRY              matter, jurisdiction and an ordered time window are fixed; a mission statement names what the case must establish
  EXIT               at least one evidence version is pinned within scope
  REQUIRED EVIDENCE  a jurisdiction code
                     an ordered time window
                     a mission statement fixed before the evidence arrives
  BLOCKING EVIDENCE  a legal hold that forbids collection in this jurisdiction
                     a scope naming a different case, which would attach this work to the wrong matter
  AUTHORITY          an actor with scoping authority; the mission may not be rewritten after evidence arrives
  OWNER              the case analyst
  FAILURE STATE      SUSPENDED
  REPLAY             timeline kind=scope_set carries the jurisdiction and mission at the tick they were fixed

=== EVIDENCE_GATHERING ===
  ENTRY              the case is scoped
  EXIT               at least one rival hypothesis is on the record
  REQUIRED EVIDENCE  every reference pins an evidence version id and a content hash
                     a source id per reference, so independence can be assessed later
  BLOCKING EVIDENCE  an unlicensed acquisition — Article 4 denies before contact, not after
                     evidence whose custody chain is broken
  AUTHORITY          an actor with acquisition authority for the source, per pkg/authz
  OWNER              the acquisition function
  FAILURE STATE      SUSPENDED
  REPLAY             timeline kind=evidence_pinned; each version re-derivable by content hash

=== HYPOTHESES_FORMED ===
  ENTRY              evidence is pinned and at least one rival explanation is recorded
  EXIT               every rival hypothesis has been tested, and claims are registered
  REQUIRED EVIDENCE  at least one rival hypothesis — a case with a single explanation has not been investigated
                     a falsifiable proposition per registered claim
  BLOCKING EVIDENCE  a hypothesis with no stated way to test it, which is a narrative rather than a rival
  AUTHORITY          an analyst; hypothesis formation is intelligence work and produces no finding
  OWNER              the intelligence function (pkg/moat/causal)
  FAILURE STATE      EVIDENCE_GATHERING
  REPLAY             timeline kinds hypothesis_recorded and hypothesis_tested carry the outcome in the tester's words

=== UNDER_QUALIFICATION ===
  ENTRY              at least one rival hypothesis exists on the record
  EXIT               every material claim carries a sealed proof object that re-verifies
  REQUIRED EVIDENCE  a reverse-proof requirement set per claim
                     an independence assessment over the source set
                     an observability verdict for every asserted absence
  BLOCKING EVIDENCE  an unresolved material contradiction — carried, never averaged away
                     a material AI contribution with no named human reviewer
                     an unassessed source set, which is UNKNOWN and never INDEPENDENT
  AUTHORITY          the qualification authority named in the proof object, under a pinned policy version
  OWNER              the Epistemic Qualification Fabric
  FAILURE STATE      EVIDENCE_GATHERING
  REPLAY             proof.VerifyHash re-derives each attached object from its components

=== RESOLVED ===
  ENTRY              every material claim is proven and every rival hypothesis tested
  EXIT               an outcome is recorded that states what was established and what was not
  REQUIRED EVIDENCE  a non-adjudicatory disposition
                     the limitations carried forward from every proof object the outcome rests on
  BLOCKING EVIDENCE  an unproven material claim
                     an untested rival hypothesis
                     an adjudicatory disposition or summary — VERIQO does not name a prevailing party
  AUTHORITY          an authority distinct from the party that generated the proof objects
  OWNER              the Case Resolution Fabric
  FAILURE STATE      UNDER_QUALIFICATION
  REPLAY             timeline kind=case_resolved carries the established and unestablished claim lists

=== SUSPENDED ===
  ENTRY              work has stopped for a stated cause
  EXIT               the cause is resolved and the case returns to the phase it can support
  REQUIRED EVIDENCE  a stated cause — suspension with no cause is indistinguishable from neglect
```

```
  BLOCKING EVIDENCE  a legal hold still in force
  AUTHORITY          any actor with case authority; suspension is always permitted
  OWNER              the case analyst
  FAILURE STATE      CLOSED
  REPLAY             timeline kind=case_suspended carries the cause at the tick it was raised

=== CLOSED ===
  ENTRY              the case is terminal for now
  EXIT               new evidence justifies reopening
  REQUIRED EVIDENCE  a stated reason for closure
  BLOCKING EVIDENCE  a retention obligation or legal hold that forbids closing the record
  AUTHORITY          an actor with case-close authority
  OWNER              the case analyst
  FAILURE STATE      CLOSED
  REPLAY             timeline kind=case_closed; the full prior record is retained

=== REOPENED ===
  ENTRY              a closed case has new evidence and a stated reason
  EXIT               the case re-enters evidence gathering or qualification
  REQUIRED EVIDENCE  a stated reason naming what is new
  BLOCKING EVIDENCE  a reopening with no new evidence, which is a re-argument rather than a reopening
  AUTHORITY          an actor with reopen authority
  OWNER              the case analyst
  FAILURE STATE      CLOSED
  REPLAY             timeline kind=case_reopened; the prior outcome is cleared but the record before it stands


=== Attestation (veriqo/pkg/platform/timestamp) ===
  OBJECT ID          chain entry hash, or the TSA's authority and serial
  SCHEMA VERSION     timestamp/v1
  CANONICALIZATION   RFC 8785 JCS via pkg/canonical/jcs; the hash field is excluded from its own computation
  CONTENT HASH       chain entries hash digest+sequence+prior+operator; external tokens are stored verbatim
  PROVENANCE         the chain operator, or the issuing authority's certificate
  AUTHORITY          the chain operator for entries; the TSA for tokens. Kind is derived by Assess, never set
  LIFECYCLE          append-only; entries are never edited or removed, because the sequence is itself evidence
  MUTATION RULE      immutable after creation; a change is a new instance with a new id, never an edit
  REPLAY             timestamp.VerifyChain recomputes every entry and its linkage

=== Breach (veriqo/pkg/insurance/obligation) ===
  OBJECT ID          BreachID
  SCHEMA VERSION     obligation/v1
  CANONICALIZATION   RFC 8785 JCS via pkg/canonical/jcs; the hash field is excluded from its own computation
  CONTENT HASH       hash over the breach and the obligation breached
  PROVENANCE         the evidence and obligation it rests on
  AUTHORITY          alleged by a party; established only by a decision-maker
  LIFECYCLE          alleged -> (established by an authority | not pursued)
  MUTATION RULE      an allegation is immutable; the authority's determination is recorded separately
  REPLAY             reproduced from the canonical audit ledger; audit.Auditor.VerifyChain re-derives it

=== Cargo (veriqo/pkg/domain/commodity) ===
  OBJECT ID          CargoID
  SCHEMA VERSION     commodity/v1
  CANONICALIZATION   RFC 8785 JCS via pkg/canonical/jcs; the hash field is excluded from its own computation
  CONTENT HASH       hash over description, quantity and quality attributes
  PROVENANCE         the shipping documents the declaration is read from
  AUTHORITY          declared by the shipper; assay results are separate evidence
  LIFECYCLE          declared -> loaded -> discharged
  MUTATION RULE      a re-declaration is a new version
  REPLAY             reproduced by recomputing the content hash from the instance's own components

=== Case (veriqo/pkg/casefabric) ===
  OBJECT ID          CaseID, scoped by tenant and opening domain
  SCHEMA VERSION     casefabric/v1
  CANONICALIZATION   RFC 8785 JCS via pkg/canonical/jcs; the hash field is excluded from its own computation
  CONTENT HASH       the timeline head hash covers the case's whole history
  PROVENANCE         the opening actor and domain, recorded in the first timeline entry
  AUTHORITY          per-phase, declared in casefabric.PhaseContracts
  LIFECYCLE          nine canonical phases; every domain state maps onto one
  MUTATION RULE      state advances only through the case engine's methods; the timeline is append-only
  REPLAY             casefabric.VerifyTimeline re-derives the hash chain

=== Causation (veriqo/pkg/insurance/causation) ===
  OBJECT ID          HypothesisID within a hypothesis set
  SCHEMA VERSION     causation/v1
  CANONICALIZATION   RFC 8785 JCS via pkg/canonical/jcs; the hash field is excluded from its own computation
  CONTENT HASH       hash over the causal chain and its supporting evidence
  PROVENANCE         the evidence versions and inference traces the chain is built from
  AUTHORITY          proposed by intelligence; never self-promoting to a finding
  LIFECYCLE          proposed -> tested -> (supported | excluded)
  MUTATION RULE      the chain is immutable; status transitions are recorded
  REPLAY             reproduced by recomputing the content hash from the instance's own components

=== Claim (veriqo/pkg/insurance/claim) ===
  OBJECT ID          ClaimID
  SCHEMA VERSION     claim/v1
  CANONICALIZATION   RFC 8785 JCS via pkg/canonical/jcs; the hash field is excluded from its own computation
  CONTENT HASH       hash over the claim and its evidence references
  PROVENANCE         the notifying party and the notification evidence version
  AUTHORITY          the claimant notifies; the insurer determines
  LIFECYCLE          notified -> ... -> closed, via casestate's fourteen states
  MUTATION RULE      the notified claim is immutable; amount changes are appended as events with their own authority checks
  REPLAY             reproduced from the canonical audit ledger; audit.Auditor.VerifyChain re-derives it

=== Clause (veriqo/pkg/domain/trade) ===
  OBJECT ID          ContractID plus clause reference
```

```
  SCHEMA VERSION     trade/v1
  CANONICALIZATION   RFC 8785 JCS via pkg/canonical/jcs; the hash field is excluded from its own computation
  CONTENT HASH       hash over the clause text and its position
  PROVENANCE         the contract version
  AUTHORITY          the parties who agreed the contract; VERIQO records the clause and does not construe it
  LIFECYCLE          immutable after creation; a change is a new instance with a new id, never an edit
  MUTATION RULE      immutable after creation; a change is a new instance with a new id, never an edit
  REPLAY             reproduced by recomputing the content hash from the instance's own components

=== Contract (veriqo/pkg/domain/trade) ===
  OBJECT ID          ContractID
  SCHEMA VERSION     trade/v1
  CANONICALIZATION   RFC 8785 JCS via pkg/canonical/jcs; the hash field is excluded from its own computation
  CONTENT HASH       hash over the contract document version
  PROVENANCE         the evidence version it is read from
  AUTHORITY          the parties; VERIQO records, it does not construe
  LIFECYCLE          DRAFT -> SUBMITTED -> FINALIZED -> (SUPERSEDED); FINALIZED is terminal for mutation
  MUTATION RULE      an amendment is a new version
  REPLAY             reproduced by recomputing the content hash from the instance's own components

=== Contradiction (veriqo/pkg/insurance/contradiction) ===
  OBJECT ID          ContradictionID
  SCHEMA VERSION     proof/v1
  CANONICALIZATION   RFC 8785 JCS via pkg/canonical/jcs; the hash field is excluded from its own computation
  CONTENT HASH       covered by the proof object's hash, including its resolved flag
  PROVENANCE         the evidence versions in conflict
  AUTHORITY          an authorized human resolves; the system only detects
  LIFECYCLE          raised -> (resolved). An unresolved material contradiction defeats sufficiency
  MUTATION RULE      the conflict itself is immutable; only the resolution may be added
  REPLAY             reproduced by recomputing the content hash from the instance's own components

=== Counterclaim (veriqo/pkg/insurance/dispute) ===
  OBJECT ID          CounterclaimID
  SCHEMA VERSION     dispute/v1
  CANONICALIZATION   RFC 8785 JCS via pkg/canonical/jcs; the hash field is excluded from its own computation
  CONTENT HASH       hash over the counterclaim and its cited evidence
  PROVENANCE         the raising party and the evidence versions they cite
  AUTHORITY          the party raising it; VERIQO attaches no weight
  LIFECYCLE          raised -> (withdrawn | determined by an authority)
  MUTATION RULE      immutable after creation; a change is a new instance with a new id, never an edit
  REPLAY             reproduced from the canonical audit ledger; audit.Auditor.VerifyChain re-derives it

=== Decision (veriqo/pkg/proof) ===
  OBJECT ID          decision hash
  SCHEMA VERSION     proof/v1
  CANONICALIZATION   RFC 8785 JCS via pkg/canonical/jcs; the hash field is excluded from its own computation
  CONTENT HASH       hash over the authorized finding, action, rationale and attributes
  PROVENANCE         Lineage() walks decision -> authorized -> finding -> proof object
  AUTHORITY          constructible only from an AuthorizedFinding; adjudicatory attributes are refused
  LIFECYCLE          immutable after creation; a change is a new instance with a new id, never an edit
  MUTATION RULE      Attributes() returns a copy, so none can be added after construction
  REPLAY             reproduced from the canonical audit ledger; audit.Auditor.VerifyChain re-derives it

=== Document (veriqo/pkg/evidence/manifest) ===
  OBJECT ID          DocumentID
  SCHEMA VERSION     manifest/v1
  CANONICALIZATION   RFC 8785 JCS via pkg/canonical/jcs; the hash field is excluded from its own computation
  CONTENT HASH       SHA-256 over the document bytes
  PROVENANCE         the acquisition record naming source, licence and acquisition path
  AUTHORITY          an actor with acquisition authority for the source (pkg/authz)
  LIFECYCLE          DRAFT -> SUBMITTED -> FINALIZED -> (SUPERSEDED); FINALIZED is terminal for mutation
  MUTATION RULE      immutable after creation; a change is a new instance with a new id, never an edit
  REPLAY             reproduced from the canonical audit ledger; audit.Auditor.VerifyChain re-derives it

=== Event (veriqo/pkg/contract/event) ===
  OBJECT ID          EventID, with sequence number assigned by the chain
  SCHEMA VERSION     event/v1
  CANONICALIZATION   RFC 8785 JCS via pkg/canonical/jcs; the hash field is excluded from its own computation
  CONTENT HASH       EventHash covers everything but itself and the signature
  PROVENANCE         ActorID and ActorType on every envelope
  AUTHORITY          Chain.Append assigns sequence and previous hash; a caller cannot supply them
  LIFECYCLE          append-only; entries are never edited or removed, because the sequence is itself evidence
  MUTATION RULE      immutable after creation; a change is a new instance with a new id, never an edit
  REPLAY             event.VerifyChain is a pure function over the envelopes

=== Evidence (veriqo/pkg/evidence/manifest) ===
  OBJECT ID          EvidenceID, unique per tenant
  SCHEMA VERSION     manifest/v1
  CANONICALIZATION   RFC 8785 JCS via pkg/canonical/jcs; the hash field is excluded from its own computation
  CONTENT HASH       SHA-256 over the raw bytes as received
  PROVENANCE         acquisition record naming source, licence and acquisition path
  AUTHORITY          an actor with acquisition authority for the source (pkg/authz)
  LIFECYCLE          DRAFT -> SUBMITTED -> FINALIZED -> (SUPERSEDED); FINALIZED is terminal for mutation
  MUTATION RULE      immutable after creation; a change is a new instance with a new id, never an edit
  REPLAY             reproduced from the canonical audit ledger; audit.Auditor.VerifyChain re-derives it

=== EvidenceVersion (veriqo/pkg/evidence/manifest) ===
  OBJECT ID          EvidenceVersionID, unique and never reused
  SCHEMA VERSION     manifest/v1
  CANONICALIZATION   RFC 8785 JCS via pkg/canonical/jcs; the hash field is excluded from its own computation
  CONTENT HASH       SHA-256 over this version's bytes
  PROVENANCE         derivation lineage back to the version it came from
  AUTHORITY          the manifest registry; SetStatus and Advance are gated on substantive prerequisites
  LIFECYCLE          DRAFT -> SUBMITTED -> FINALIZED -> (SUPERSEDED); FINALIZED is terminal for mutation
  MUTATION RULE      immutable once FINALIZED; the custody chain head freezes with it
```

```
  REPLAY             pkg/replay ManifestAdapter restores state from the record

=== Fact (veriqo/pkg/evidence/semantics) ===
  OBJECT ID          FactID
  SCHEMA VERSION     semantics/v1
  CANONICALIZATION   RFC 8785 JCS via pkg/canonical/jcs; the hash field is excluded from its own computation
  CONTENT HASH       hash over the asserted fact and its evidence version references
  PROVENANCE         the evidence versions the fact is read from
  AUTHORITY          the extraction pipeline; no AI may create one
  LIFECYCLE          immutable after creation; a change is a new instance with a new id, never an edit
  MUTATION RULE      immutable after creation; a change is a new instance with a new id, never an edit
  REPLAY             reproduced by recomputing the content hash from the instance's own components

=== Finding (veriqo/pkg/proof) ===
  OBJECT ID          finding hash
  SCHEMA VERSION     proof/v1
  CANONICALIZATION   RFC 8785 JCS via pkg/canonical/jcs; the hash field is excluded from its own computation
  CONTENT HASH       hash over the proof hash, proposition, stance, qualification and limitations
  PROVENANCE         the sealed proof object it derives from, re-verified at construction
  AUTHORITY          produced only from a sufficient object; adopted only by an authority who is not the author
  LIFECYCLE          derived -> authorized. The zero value cannot be authorized
  MUTATION RULE      all fields unexported; every accessor returns a copy
  REPLAY             reproduced by recomputing the content hash from the instance's own components

=== Hypothesis (veriqo/pkg/moat/causal) ===
  OBJECT ID          HypothesisID
  SCHEMA VERSION     causal/v1
  CANONICALIZATION   RFC 8785 JCS via pkg/canonical/jcs; the hash field is excluded from its own computation
  CONTENT HASH       hash over the hypothesis and its cited inference trace
  PROVENANCE         pkg/inference.InferenceTrace pins every input
  AUTHORITY          intelligence proposes; a hypothesis can never become a finding inside this fabric
  LIFECYCLE          proposed -> tested. A case cannot resolve with an untested rival
  MUTATION RULE      the statement is immutable; the test outcome is appended
  REPLAY             re-derivable from the same evidence via the trace

=== Loss (veriqo/pkg/insurance/quantum) ===
  OBJECT ID          LossID
  SCHEMA VERSION     quantum/v1
  CANONICALIZATION   RFC 8785 JCS via pkg/canonical/jcs; the hash field is excluded from its own computation
  CONTENT HASH       hash over the loss and its evidence backing
  PROVENANCE         the evidence-backed amounts it aggregates
  AUTHORITY          computed, never asserted; every amount cites its evidence
  LIFECYCLE          estimated -> quantified -> (settled)
  MUTATION RULE      a revision is a new calculation with its own id
  REPLAY             reproduced by recomputing the content hash from the instance's own components

=== Model (veriqo/pkg/ai/gateway) ===
  OBJECT ID          model id plus version
  SCHEMA VERSION     gateway/v1
  CANONICALIZATION   RFC 8785 JCS via pkg/canonical/jcs; the hash field is excluded from its own computation
  CONTENT HASH       contribution records hash the prompt and output
  PROVENANCE         provider, version and training permission
  AUTHORITY          an allowlist that fails closed when empty
  LIFECYCLE          approved -> (withdrawn)
  MUTATION RULE      a version change is a different model, never an update
  REPLAY             contribution hashes are recomputable from the recorded prompt and output

=== NextBestEvidence (veriqo/pkg/qualification/nextbest) ===
  OBJECT ID          CandidateID
  SCHEMA VERSION     nextbest/v1
  CANONICALIZATION   RFC 8785 JCS via pkg/canonical/jcs; the hash field is excluded from its own computation
  CONTENT HASH       covered by the direction's proof hash
  PROVENANCE         the insufficient proof object that produced the direction
  AUTHORITY          hard rights gates run before scoring; a denied candidate is excluded, never ranked
  LIFECYCLE          proposed -> (pursued | excluded). What was not pursued, and why, stays on the record
  MUTATION RULE      immutable after creation; a change is a new instance with a new id, never an edit
  REPLAY             nextbest.Rank is deterministic with an id tie-break: the same candidates reproduce the same ranking

=== Obligation (veriqo/pkg/insurance/obligation) ===
  OBJECT ID          ObligationID
  SCHEMA VERSION     obligation/v1
  CANONICALIZATION   RFC 8785 JCS via pkg/canonical/jcs; the hash field is excluded from its own computation
  CONTENT HASH       hash over the obligation and its source clause
  PROVENANCE         the contract clause and version the obligation is read from
  AUTHORITY          the contract; VERIQO identifies, it does not impose
  LIFECYCLE          open -> (discharged | breached)
  MUTATION RULE      the obligation text is immutable; status transitions are appended as events, never edited in place
  REPLAY             reproduced from the canonical audit ledger; audit.Auditor.VerifyChain re-derives it

=== Organization (veriqo/pkg/insurance/party) ===
  OBJECT ID          OrganizationID
  SCHEMA VERSION     party/v1
  CANONICALIZATION   RFC 8785 JCS via pkg/canonical/jcs; the hash field is excluded from its own computation
  CONTENT HASH       hash over legal identity
  PROVENANCE         the onboarding record and the registry extract the identity was verified against
  AUTHORITY          the participant registry, which is the sole creator of these identities
  LIFECYCLE          registered -> (dissolved)
  MUTATION RULE      identity immutable; attributes versioned
  REPLAY             reproduced from the canonical audit ledger; audit.Auditor.VerifyChain re-derives it

=== Party (veriqo/pkg/insurance/party) ===
  OBJECT ID          PartyID
  SCHEMA VERSION     party/v1
  CANONICALIZATION   RFC 8785 JCS via pkg/canonical/jcs; the hash field is excluded from its own computation
  CONTENT HASH       hash over identity and role assignments
```

```
  PROVENANCE         the onboarding record and the registry extract the identity was verified against
  AUTHORITY          the participant registry, which is the sole creator of these identities
  LIFECYCLE          invited -> active -> (suspended | withdrawn)
  MUTATION RULE      role changes are events, never in-place edits
  REPLAY             reproduced from the canonical audit ledger; audit.Auditor.VerifyChain re-derives it

=== Person (veriqo/pkg/insurance/party) ===
  OBJECT ID          PersonID
  SCHEMA VERSION     party/v1
  CANONICALIZATION   RFC 8785 JCS via pkg/canonical/jcs; the hash field is excluded from its own computation
  CONTENT HASH       hash over identity attributes
  PROVENANCE         the onboarding record and the registry extract the identity was verified against
  AUTHORITY          the participant registry, which is the sole creator of these identities
  LIFECYCLE          registered -> (withdrawn)
  MUTATION RULE      identity immutable; attributes versioned. Erasure never edits in place -- it goes through pkg/governance/data
  REPLAY             reproduced from the canonical audit ledger; audit.Auditor.VerifyChain re-derives it

=== Policy (veriqo/pkg/insurance/coverage) ===
  OBJECT ID          PolicyID
  SCHEMA VERSION     coverage/v1
  CANONICALIZATION   RFC 8785 JCS via pkg/canonical/jcs; the hash field is excluded from its own computation
  CONTENT HASH       hash over the policy wording version
  PROVENANCE         the policy document evidence version the wording is read from
  AUTHORITY          the insurer, whose wording it is; VERIQO does not construe it
  LIFECYCLE          DRAFT -> SUBMITTED -> FINALIZED -> (SUPERSEDED); FINALIZED is terminal for mutation
  MUTATION RULE      an endorsement is a new version
  REPLAY             reproduced by recomputing the content hash from the instance's own components

=== Port (veriqo/pkg/domain/maritime) ===
  OBJECT ID          UN/LOCODE
  SCHEMA VERSION     maritime/v1
  CANONICALIZATION   RFC 8785 JCS via pkg/canonical/jcs; the hash field is excluded from its own computation
  CONTENT HASH       hash over the port reference data version
  PROVENANCE         the reference source and the version of it that was loaded
  AUTHORITY          reference data, not evidence; used to resolve, never to establish
  LIFECYCLE          DRAFT -> SUBMITTED -> FINALIZED -> (SUPERSEDED); FINALIZED is terminal for mutation
  MUTATION RULE      immutable after creation; a change is a new instance with a new id, never an edit
  REPLAY             reproduced by recomputing the content hash from the instance's own components

=== ProofObject (veriqo/pkg/proof) ===
  OBJECT ID          CanonicalHash
  SCHEMA VERSION     proof/v1
  CANONICALIZATION   RFC 8785 JCS via pkg/canonical/jcs; the hash field is excluded from its own computation
  CONTENT HASH       JCS hash over all twenty-three components, excluding the hash and signature
  PROVENANCE         Provenance component: generator, pipeline version and pinned input hashes
  AUTHORITY          the Authority component, with a pinned policy version
  LIFECYCLE          created -> sealed -> (externally attested). Levels derive; none is settable
  MUTATION RULE      immutable after sealing; Seal overwrites any author-supplied stance or sufficiency
  REPLAY             proof.VerifyHash recomputes the hash from the components

=== ProofObligation (veriqo/pkg/qualification/reverseproof) ===
  OBJECT ID          RequirementID within a claim's requirement set
  SCHEMA VERSION     reverseproof/v1
  CANONICALIZATION   RFC 8785 JCS via pkg/canonical/jcs; the hash field is excluded from its own computation
  CONTENT HASH       covered by the proof object's hash
  PROVENANCE         the claim and condition it derives from
  AUTHORITY          the analyst who built the set; a requirement with no falsifying observation is refused
  LIFECYCLE          unattempted -> obtained | observed-absent | unobtainable
  MUTATION RULE      the expectation is fixed in advance so a later observation cannot be reinterpreted to fit
  REPLAY             reverseproof.Analyze is pure: re-running it over the same set reproduces the gap exactly

=== Proposition (veriqo/pkg/proof) ===
  OBJECT ID          PropositionID
  SCHEMA VERSION     proof/v1
  CANONICALIZATION   RFC 8785 JCS via pkg/canonical/jcs; the hash field is excluded from its own computation
  CONTENT HASH       covered by the proof object's canonical hash
  PROVENANCE         the case and actor that registered it
  AUTHORITY          an analyst; a proposition must be falsifiable or registration is refused
  LIFECYCLE          immutable after creation; a change is a new instance with a new id, never an edit
  MUTATION RULE      immutable after creation; a change is a new instance with a new id, never an edit
  REPLAY             reproduced by recomputing the content hash from the instance's own components

=== Qualification (veriqo/pkg/qualification/state) ===
  OBJECT ID          ClaimID plus policy version
  SCHEMA VERSION     qualification/v1
  CANONICALIZATION   RFC 8785 JCS via pkg/canonical/jcs; the hash field is excluded from its own computation
  CONTENT HASH       covered by the proof object carrying it
  PROVENANCE         the rationale, the tick and the evidence set the verdict was reached over
  AUTHORITY          the qualification authority under a pinned policy
  LIFECYCLE          one of ten states; there is no PROVEN state and Parse refuses one by name
  MUTATION RULE      a new qualification supersedes; material dissent is carried, never deleted
  REPLAY             a pure function of its inputs; re-running reproduces the state

=== Quantum (veriqo/pkg/insurance/quantum) ===
  OBJECT ID          CalculationID
  SCHEMA VERSION     quantum/v1
  CANONICALIZATION   RFC 8785 JCS via pkg/canonical/jcs; the hash field is excluded from its own computation
  CONTENT HASH       hash over the calculation inputs and result
  PROVENANCE         the evidence-backed inputs
  AUTHORITY          deterministic computation; no discretionary adjustment without an authority record
  LIFECYCLE          immutable after creation; a change is a new instance with a new id, never an edit
  MUTATION RULE      immutable after creation; a change is a new instance with a new id, never an edit
  REPLAY             reproduced by recomputing the content hash from the instance's own components

=== Resolution (veriqo/pkg/casefabric) ===
```

```
  OBJECT ID          CaseID plus resolution tick
  SCHEMA VERSION     casefabric/v1
  CANONICALIZATION   RFC 8785 JCS via pkg/canonical/jcs; the hash field is excluded from its own computation
  CONTENT HASH       covered by the timeline entry that records it
  PROVENANCE         the resolving authority
  AUTHORITY          an authority distinct from the proof author
  LIFECYCLE          produced once per resolution; cleared on reopening, with the prior record retained
  MUTATION RULE      limitations may be added; established and unestablished claim lists are computed, never supplied
  REPLAY             reproduced from the canonical audit ledger; audit.Auditor.VerifyChain re-derives it

=== ResolutionPackage (veriqo/pkg/insurance/casepack) ===
  OBJECT ID          PackageID
  SCHEMA VERSION     casepack/v1
  CANONICALIZATION   RFC 8785 JCS via pkg/canonical/jcs; the hash field is excluded from its own computation
  CONTENT HASH       manifest hash over every included artefact
  PROVENANCE         the case and the proof objects it is assembled from
  AUTHORITY          the case authority
  LIFECYCLE          mutable until sealed, immutable after; the seal computes the content hash and any later edit breaks it
  MUTATION RULE      mutable until sealed, immutable after; the seal computes the content hash and any later edit breaks it
  REPLAY             the package verifies standalone via the independent verifier

=== Responsibility (veriqo/pkg/insurance/causation) ===
  OBJECT ID          ResponsibilityID
  SCHEMA VERSION     causation/v1
  CANONICALIZATION   RFC 8785 JCS via pkg/canonical/jcs; the hash field is excluded from its own computation
  CONTENT HASH       hash over the attribution and its basis
  PROVENANCE         the causal hypothesis it rests on
  AUTHORITY          attributed as an analytical position, never as a legal determination
  LIFECYCLE          attributed -> (accepted | disputed | determined by an authority)
  MUTATION RULE      immutable after creation; a change is a new instance with a new id, never an edit
  REPLAY             reproduced from the canonical audit ledger; audit.Auditor.VerifyChain re-derives it

=== Shipment (veriqo/pkg/domain/supplychain) ===
  OBJECT ID          ShipmentID
  SCHEMA VERSION     supplychain/v1
  CANONICALIZATION   RFC 8785 JCS via pkg/canonical/jcs; the hash field is excluded from its own computation
  CONTENT HASH       hash over the shipment and its leg references
  PROVENANCE         the transport documents the shipment is constructed from
  AUTHORITY          the parties to the carriage, whose declarations VERIQO records
  LIFECYCLE          booked -> in transit -> delivered
  MUTATION RULE      each leg is appended
  REPLAY             reproduced from the canonical audit ledger; audit.Auditor.VerifyChain re-derives it

=== Timeline (veriqo/pkg/casefabric) ===
  OBJECT ID          CaseID plus sequence number
  SCHEMA VERSION     casefabric/v1
  CANONICALIZATION   RFC 8785 JCS via pkg/canonical/jcs; the hash field is excluded from its own computation
  CONTENT HASH       each entry hashes its content and its predecessor
  PROVENANCE         the actor and tick on every entry
  AUTHORITY          written only by the case engine
  LIFECYCLE          append-only; entries are never edited or removed, because the sequence is itself evidence
  MUTATION RULE      never edited; a correction is a new entry
  REPLAY             casefabric.VerifyTimeline

=== Transaction (veriqo/pkg/domain/trade) ===
  OBJECT ID          TransactionID
  SCHEMA VERSION     trade/v1
  CANONICALIZATION   RFC 8785 JCS via pkg/canonical/jcs; the hash field is excluded from its own computation
  CONTENT HASH       hash over the transaction terms
  PROVENANCE         the instrument or instruction it derives from
  AUTHORITY          the transacting parties
  LIFECYCLE          instructed -> settled -> (reversed)
  MUTATION RULE      a reversal is a new transaction referencing the original
  REPLAY             reproduced from the canonical audit ledger; audit.Auditor.VerifyChain re-derives it

=== Vessel (veriqo/pkg/domain/maritime) ===
  OBJECT ID          IMO number where known, otherwise a VERIQO vessel id
  SCHEMA VERSION     maritime/v1
  CANONICALIZATION   RFC 8785 JCS via pkg/canonical/jcs; the hash field is excluded from its own computation
  CONTENT HASH       hash over identity attributes at a point in time
  PROVENANCE         the registry or source the identity is read from
  AUTHORITY          identity resolution below threshold stays unresolved rather than merging two vessels
  LIFECYCLE          registered -> (renamed | reflagged | scrapped), each a new version
  MUTATION RULE      identity attributes are versioned, never overwritten
  REPLAY             reproduced by recomputing the content hash from the instance's own components

=== Voyage (veriqo/pkg/domain/maritime) ===
  OBJECT ID          VoyageID
  SCHEMA VERSION     maritime/v1
  CANONICALIZATION   RFC 8785 JCS via pkg/canonical/jcs; the hash field is excluded from its own computation
  CONTENT HASH       hash over the voyage and its port calls
  PROVENANCE         the position and port-call sources
  AUTHORITY          constructed from evidence; a gap in coverage is an observability question, not an inference
  LIFECYCLE          planned -> underway -> completed
  MUTATION RULE      corrections are new versions
  REPLAY             reproduced by recomputing the content hash from the instance's own components
```